

Learn Low-Level Design in 25 Days: Day 24—Case Study: Designing a Movie Ticket Booking System

The Real-World Story

TechPreneur

Published November 6, 2025

Free: No

Read on Freedium · original

Contents

Core Concepts: How Does a Movie Booking System Work? 0

1. The Core Entities 0

2. Problem: Avoiding Double-Booking 0

3. Key Classes and Relationships 0

4. Handling Concurrency — The Seat Lock Manager 0

5. Booking Flow — End to End 0

6. Extending the System 0

Visual Summary 0

Try It Yourself 0

Summary & Next-Day Teaser 0

Key Takeaways 0

When to Use 0

When to Avoid 0

Imagine it's Friday night. You're scrolling through your phone, trying to book seats for the latest blockbuster. You pick a theatre, choose your showtime, grab two seats in the middle row, and pay — all within seconds.

Now pause for a moment. Behind that sleek UI is a complex orchestration of systems ensuring that:

- You don't double-book a seat someone else just grabbed.
- Seats are locked for a few minutes during checkout.
- Theatres, movies, and showtimes stay perfectly in sync.

Welcome to **Day 24 of **Learn Low-Level Design in 25 Days** — today, we're doing a full-fledged **case study: designing a Movie Ticket Booking System**.



By the end of this session, you'll know how to:

- Model real-world entities like Movies, Theatres, Shows, and Seats.
- Handle concurrency and seat-locking.
- Design a system flexible enough to handle multiple cities, languages, and multiplexes.

This is the kind of design you'll face in interviews — and even more importantly, on real projects.

Core Concepts: How Does a Movie Booking System Work?

Before jumping into code, let's break the problem down into key modules — the way a senior engineer would think through it.

1. The Core Entities

At the heart of the system, we have a few primary entities:

- **Movie** — represents the film itself (title, duration, language, genre).
- **Theatre** — physical location with multiple screens.
- **Screen** — holds shows for different movies.
- **Show** — a specific screening of a movie at a certain time and screen.
- **Seat** — individual spots in a screen.
- **Booking** — the record of a user reserving seats for a show.

Think of it like this:

```
City → Theatres → Screens → Shows → Seats → Booking
```

That hierarchy ensures modularity — a change in one theatre doesn't break the rest of the system.

2. Problem: Avoiding Double-Booking

Here's the key design challenge — **seat availability**. If two users click "Book" for the same seat at the same time, the system must prevent overlap.

The naive (and wrong) approach:

- Mark the seat as "booked" only after payment succeeds. Result: Two users might pay for the same seat. Disaster.

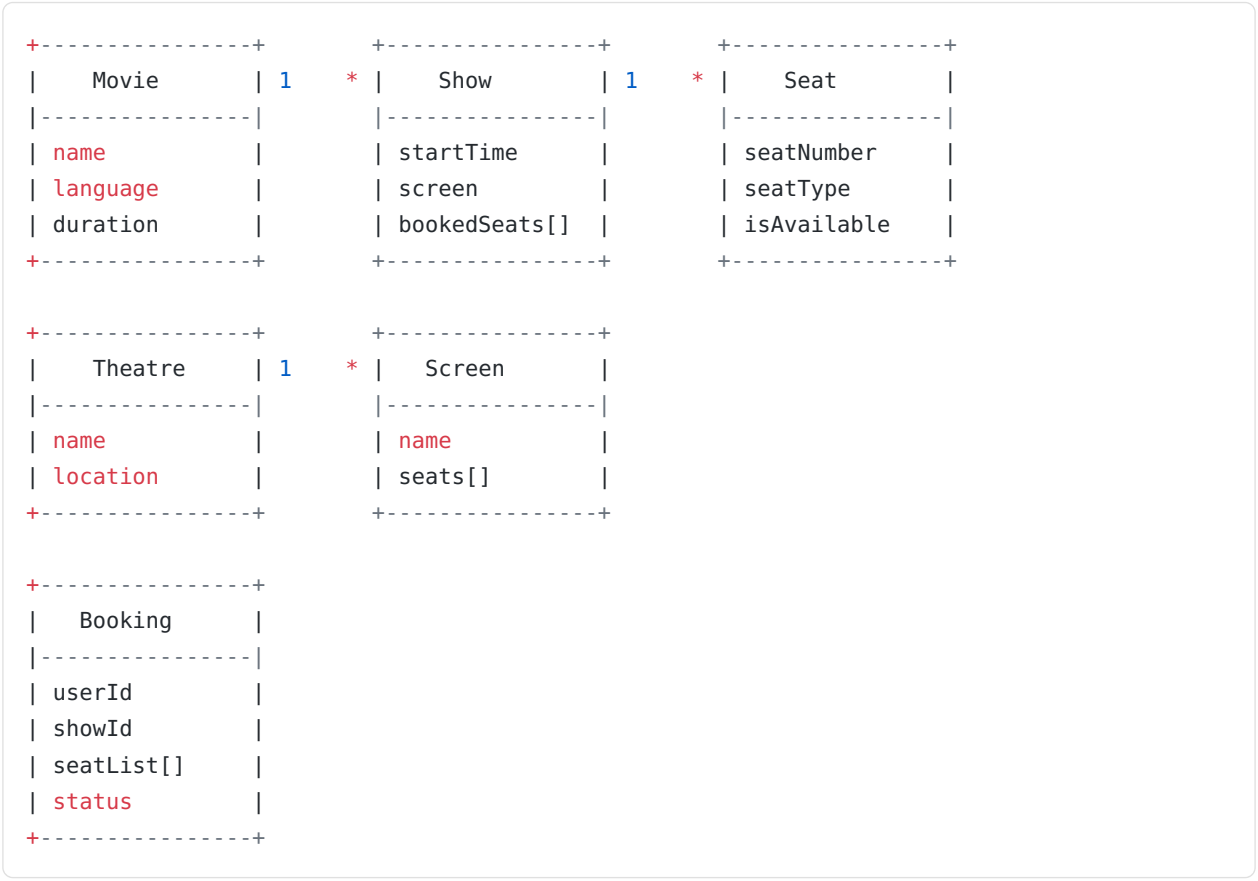
The right approach:

- **Introduce a temporary "seat lock."** When a user selects seats, the system locks them for a few minutes. If payment completes, they become "booked." If not, the lock expires.

This is similar to how an online checkout cart works — your items are "reserved" but not truly yours until you pay.

3. Key Classes and Relationships

Let's sketch a minimal class diagram.



4. Handling Concurrency — The Seat Lock Manager

To avoid booking conflicts, we introduce a **SeatLockManager**.

Responsibilities:

- Lock seats for a short duration.
- Prevent others from booking locked seats.
- Release locks automatically if payment fails or timeout occurs.

```
class SeatLockManager {
    Map<Seat, SeatLock> locks = new HashMap<>();

    public void lockSeats(List<Seat> seats, Show show, String user, int timeout) {
        for (Seat seat : seats) {
            if (isSeatLocked(seat, show)) {
                throw new SeatAlreadyLockedException();
            }
            locks.put(seat, new SeatLock(seat, show, user, timeout));
        }
    }

    public boolean isSeatLocked(Seat seat, Show show) {
        SeatLock lock = locks.get(seat);
        if (lock == null) return false;
        if (lock.isExpired()) {
            locks.remove(seat);
            return false;
        }
        return true;
    }
}
```

The `SeatLock` object holds details like lock time, expiry, and user. This pattern encapsulates concurrency neatly and keeps our design clean.

5. Booking Flow — End to End

Here's a simplified flow:

```
User → selects Show & Seats
    → SeatLockManager.lockSeats()
    → PaymentService.processPayment()
        → if success → BookingService.confirmBooking()
        → if fail → SeatLockManager.releaseSeats()
```

The **BookingService** becomes the central coordinator — it talks to both the **SeatLockManager** and **PaymentService**.

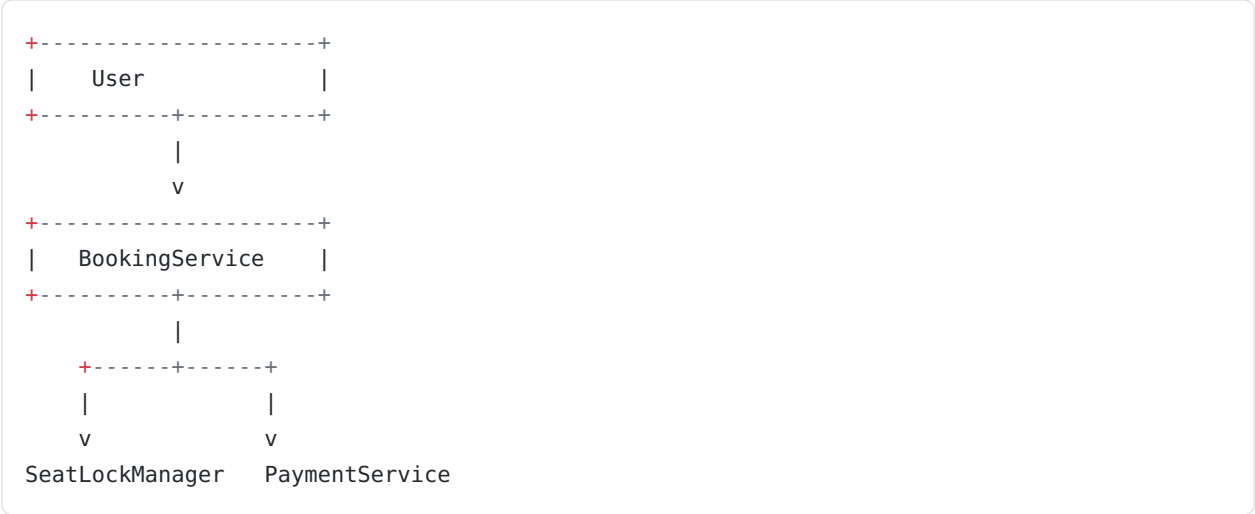
6. Extending the System

Once the core design is solid, you can easily add:

- **SearchService** (find movies by city, genre, language)
- **NotificationService** (email or SMS after booking)
- **AnalyticsService** (track popular shows or theatres)

That's the beauty of clean, object-oriented design — you can keep layering features without rewriting the base.

Visual Summary



Try It Yourself

1. **Challenge:** Implement a `SeatLockManager` that releases expired locks automatically after a timeout. Hint: Use a background thread or scheduled executor.
2. **Reflection:** What would change if this were a **concert booking system** with dynamic seat pricing?

Summary & Next-Day Teaser

Key Takeaways

- Model entities around real-world concepts (Movie, Show, Seat, Booking).
- Always handle concurrency explicitly — seat locking is crucial.
- Design for extensibility — add new services without breaking core logic.
- Think in modules: Booking, Search, Payment, Notifications.

When to Use

- Any system with limited inventory (tickets, hotel rooms, flight seats).

When to Avoid

- Systems where availability isn't real-time (e.g., content streaming).

Tomorrow, we'll wrap up the series with **Day 25 — Building Your Own LLD Portfolio for Interviews**, where you'll learn how to document, present, and showcase these designs like a pro.

You can follow and save the full **25-day series here:** Learn Low-Level Design in 25 Days: A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)